

Milestone 4 | Git & GitHub | Interview Questions

Q1. What is version control? Why do developers use it?

Answer: Version control is a system that tracks every change made to your code over time. It saves a full history of what changed, who changed it, and when.

Without version control:

- You might accidentally overwrite working code
- You cannot go back to a previous version if something breaks
- Two developers working on the same file will create conflicts with no way to resolve them

With version control:

- Full history of every change is saved
- You can go back to any previous version (rollback)
- Multiple developers can work on the same project safely
- Every change is linked to a person and a reason (commit message)

Real analogy: Think of it like Google Docs version history — you can see every edit ever made and go back to any older version at any time.

Interview Tip: Every company uses version control. Not knowing Git is a dealbreaker in most developer interviews, including at TCS, Infosys, Wipro, Zoho, and all startups.

Q2. What is the difference between a centralized and a distributed version control system?

Answer:

Feature	Centralized VCS	Distributed VCS
Where code lives	One central server	Every developer has full copy
Work offline?	✗ No — needs server connection	✓ Yes — full repo is local

Feature	Centralized VCS	Distributed VCS
Risk	Server fails = no one can work	Each developer is a full backup
Example	SVN (Subversion), CVS	Git, Mercurial

Git is a distributed VCS. This means every developer's machine has the complete project history. Even if the central server goes down, no history is lost.

Interview Tip: Interviewers often ask "what kind of VCS is Git?" — the answer is **distributed**.

Q3. What is Git? How is it different from GitHub?

Answer:

	Git	GitHub
What is it?	Version control tool that runs on your computer	Website that hosts Git repositories online
Works offline?	✅ Yes	❌ No — needs internet
Made by	Linus Torvalds (2005)	GitHub Inc. (now owned by Microsoft)
Purpose	Track changes locally	Share, collaborate, back up code online

```
Shell
# Git - runs on your computer
git init
git commit -m "first commit"

# GitHub - you push your local Git repo to GitHub
git push origin main
```

Simple way to remember: Git is the tool. GitHub is the website that stores your Git projects online — like how WhatsApp is the app and your phone is the device it runs on.

Q4. What are the three main areas in Git? Explain each one.

Answer: Git works with three distinct areas:

- 1. Working Directory** This is your actual project folder — the files you see and edit on your computer. Any new file or change you make starts here.
- 2. Staging Area (Index)** This is a preparation zone. You move files here when you decide they are ready to be saved. Only staged files go into the next commit.
- 3. Repository (.git folder)** This is where Git permanently saves your history. Every commit is stored here as a snapshot.

None

```
Working Directory → git add → Staging Area → git commit → Repository  
(your files)                (ready files)                (saved history)
```

Real analogy: Think of ordering food at a restaurant.

- Working Directory = items on the menu you are looking at
- Staging Area = items added to your cart/order
- Repository = the final confirmed order sent to the kitchen

Q5. What is `git init`? What does it do?

Answer: `git init` initializes a new Git repository in your current folder. It creates a hidden `.git` folder that stores all version control information.

Shell

```
# Step 1: Go to your project folder  
cd my-project  
  
# Step 2: Initialize Git  
git init  
# Output: Initialized empty Git repository in /my-project/.git/
```

What happens:

- A `.git` hidden folder is created inside your project
- Git starts tracking this folder as a repository
- Your files are not yet tracked — you still need to `git add` and `git commit`

Common Mistake: Running `git init` inside the wrong folder, like your Desktop or home directory. Always navigate into your project folder first.

Q6. What is `git status`? When should you use it?

Answer: `git status` shows the current state of your working directory and staging area. It tells you which files are untracked, modified, or staged.

```
Shell
git status
```

Possible states shown:

- `Untracked files` — new files Git has never seen
- `Changes not staged for commit` — modified files not yet added to staging
- `Changes to be committed` — files in staging, ready to commit
- `nothing to commit, working tree clean` — everything is saved

Interview Tip: Always run `git status` before and after every Git command when you are starting out. It is your "what is happening right now" command.

Q7. What is `git add`? What is the difference between `git add <file>` and `git add .`?

Answer: `git add` moves files from the working directory into the staging area — preparing them for a commit.

```
Shell
# Add a specific file only
git add index.html

# Add all changed files at once
git add .
```

```
# Add all files in a specific folder
git add src/
```

Command	What it stages
<code>git add index.html</code>	Only <code>index.html</code>
<code>git add .</code>	All new and modified files in current directory
<code>git add -A</code>	All changes including deletions

When to use which:

- Use `git add <file>` when you want to stage only specific files for a focused commit
- Use `git add .` when all your changes belong together in one commit

Q8. What is a commit in Git? What makes a good commit message?

Answer: A commit is a permanent snapshot of your staged changes saved to the repository. Every commit has a unique ID, author name, date, and a message describing what changed.

```
Shell
git commit -m "Add login form with email and password fields"
```

What makes a good commit message:

```
Shell
# ❌ Bad commit messages
git commit -m "done"
git commit -m "changes"
git commit -m "fix"

# ✅ Good commit messages
git commit -m "Add user login form with validation"
git commit -m "Fix navbar not showing on mobile screens"
git commit -m "Remove console.log from payment module"
```

Good commit messages:

- Are short but descriptive (under 72 characters)
- Start with a verb: Add, Fix, Update, Remove, Refactor
- Explain *what* changed and *why*, not *how*

Interview Tip: Companies like Zoho and Infosys ask about commit message best practices in code review discussions. Bad commit messages are flagged in real pull request reviews.

Q9. A fresher at a startup added all their files with `git add .` but then realized one file has errors. How do they remove just that file from staging without losing their changes?

Answer: Use `git restore --staged <file>` (modern Git) or `git reset HEAD <file>` (older Git).

Shell

```
# ✅ Modern approach – unstage a specific file
git restore --staged broken-file.js
```

```
# ✅ Older approach – same effect
git reset HEAD broken-file.js
```

After running this, the file goes back to the working directory (modified but unstaged). Your changes are not lost — the file is just removed from the next commit.

Shell

```
# Check the result
git status
# broken-file.js will now show as "Changes not staged for commit"
```

Q10. What is `git log`? What information does it show?

Answer: `git log` shows the full commit history of your repository — newest commit first.

```
Shell
git log
```

Output looks like:

```
None

commit a3f9b21c4d8e1f6789abc123def456789012345
Author: Rahul Sharma <rahul@example.com>
Date:   Mon Apr 7 10:32:14 2025 +0530

    Add responsive navbar using Flexbox

commit 7bc1d45e9f2a3b4c5d6e7f8901234567890abcd
Author: Rahul Sharma <rahul@example.com>
Date:   Sun Apr 6 16:45:00 2025 +0530

    Initial project setup with HTML structure
```

Useful `git log` options:

```
Shell

git log --oneline          # one line per commit - compact view
git log --oneline --graph  # shows branch/merge history visually
git log -5                 # shows only the last 5 commits
```

Interview Tip: `git log --oneline` is used constantly in real development to quickly check what was committed recently.

Q11. What is a `.gitignore` file? Why is it important?

Answer: `.gitignore` is a file that tells Git which files and folders to never track or include in commits.

Why you need it:

- `node_modules/` folder can have 50,000+ files — no one needs this in the repository, it gets reinstalled with `npm install`

- `.env` files contain passwords, API keys, database URLs — these should never be uploaded to GitHub
- Build output folders like `dist/` or `build/` are generated files, not source code

Shell

```
# Create a .gitignore file
touch .gitignore
```

None

```
# Inside .gitignore
```

```
node_modules/
.env
.env.local
dist/
build/
*.log
.DS_Store
```

Shell

```
# After creating .gitignore
git status
# node_modules/ and .env will no longer appear as untracked files
```

Interview Tip: Pushing `.env` files with passwords to a public GitHub repo is a very serious security mistake. Companies lose lakhs of rupees because of leaked API keys. Always add `.env` to `.gitignore`.

Q12. What is the difference between `git add .` and `git commit`? Why are there two separate steps?

Answer: These are two separate steps by design — they give you control over exactly what gets saved.

Command	What it does
<code>git add .</code>	Moves files to staging — "I am planning to save these"
<code>git commit -m ""</code>	Permanently saves staged files — "I am saving now"

Why two steps? Imagine you made changes to 5 files but only 3 are ready. You can `git add` only those 3, commit them with one message, then `git add` the remaining 2 later with a different message.

Shell

```
#  Two separate commits for two separate features
git add navbar.html navbar.css
git commit -m "Add responsive navbar"

git add login.html login.css
git commit -m "Add login page"
```

Interview Tip: A common interview question — "why does Git have a staging area?" The answer is: it gives you fine-grained control over what goes into each commit.

Q13. What does it mean when `git status` shows a file as "untracked"?

Answer: An untracked file is a new file that Git has never seen before — it has never been added or committed. Git knows the file exists in your folder, but it is not watching or recording changes to it.

Shell

```
# You create a new file
touch about.html

# Git status shows it as untracked
git status
# Output:
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#       about.html
```

To start tracking it:

```
Shell
git add about.html
git commit -m "Add about page"
```

Once a file has been committed at least once, Git always tracks it from that point on.

Q14. How do you set up Git for the first time on a new computer?

Answer: You need to configure your name and email once. Git uses this to identify who made each commit.

```
Shell
# Set your name
git config --global user.name "Priya Sharma"

# Set your email (same as your GitHub account email)
git config --global user.email "priya.sharma@example.com"

# Check all settings
git config --list
```

The `--global` flag means this setting applies to all your Git projects on this computer. You only need to do this once per machine.

Interview Tip: In team environments, this is important because every commit shows the author's name and email in `git log`. If this is not set correctly, your contributions are not properly attributed.

Q15. What is a Git branch? Why do developers use branches?

Answer: A branch is a separate, independent line of development. It lets you work on a new feature or fix without touching the main working code.

Real analogy: Imagine the main project is a published book. A branch is like a draft copy where you write a new chapter. Once the chapter is reviewed and approved, you add it to the

published book. If you don't like the chapter, you throw the draft away — the published book is unaffected.

Why use branches:

- Work on a new feature without breaking working code
- Multiple developers can work in parallel on different features
- Easy to discard experimental work if it does not work out

Shell

```
# main branch = stable, working code
# feature branches = new features in progress

git branch feature-login      # create branch
git switch feature-login      # switch to it
# ... make changes, commit them ...
git switch main               # go back to main
git merge feature-login       # bring changes into main
```

Q16. What is the default branch in Git? Has its name changed recently?

Answer: Historically, the default branch was called `master`. In recent years, the industry and GitHub moved to calling it `main` as the standard default name.

Shell

```
# Check what branches exist and which is current
git branch
# * main      ← asterisk shows current branch

# When you run git init, the default branch is now 'main' in most setups
```

Interview Tip: If you see `master` in older projects and `main` in newer ones — both are just default branch names. The behavior is identical. Some companies use `master`, some use `main`. Know both.

Q17. How do you create a new branch and switch to it?

Answer:

Shell

```
# Method 1 - Two steps
git branch feature-navbar    # creates the branch
git switch feature-navbar    # switches to it

# Method 2 - One step (create and switch at once)
git switch -c feature-navbar # -c means create

# Older method (still works)
git checkout -b feature-navbar
```

Check which branch you are on:

Shell

```
git branch
# Output:
# * feature-navbar  ← you are here
#   main
```

Interview Tip: `git switch` was introduced in Git 2.23 as a cleaner alternative to `git checkout` for switching branches. Both work, but interviewers may ask about `git switch` to check if you know modern Git.

Q18. A developer at Zoho was working on a new feature branch and made several commits. How do they bring those changes into the main branch?

Answer: Use `git merge` after switching back to the main branch.

Shell

```
# Step 1: Make sure all changes on the feature branch are committed
git status    # should be clean

# Step 2: Switch to main branch
git switch main

# Step 3: Merge the feature branch into main
git merge feature-payment
```

```
# Step 4: Verify the merge
git log --oneline
```

What happens during merge:

- Git looks at the commit history of both branches
- Combines the changes together
- Creates a merge commit (unless it's a fast-forward merge)

Q19. What is a fast-forward merge in Git?

Answer: A fast-forward merge happens when the main branch has not changed since the feature branch was created. Git simply moves the main branch pointer forward to the latest commit of the feature branch — no new merge commit is created.

None

Before merge:

main: A → B

feature: → C → D

After fast-forward merge:

main: A → B → C → D

Shell

```
git switch main
```

```
git merge feature-ui
```

```
# Output: Fast-forward
```

When it does NOT fast-forward: If someone committed to `main` while you were working on your feature branch, Git creates a merge commit instead.

Q20. What is a merge conflict in Git? When does it happen?

Answer: A merge conflict happens when two branches have changed the **same line** of the same file. Git does not know which version to keep, so it stops and asks you to decide manually.

When it happens:

- Two developers edit the same line in the same file
- One developer edits a file that another deleted
- Two branches both modify the same function

Shell

```
# Trying to merge causes a conflict
git merge feature-contact
# Output:
# CONFLICT (content): Merge conflict in index.html
# Automatic merge failed; fix conflicts and then commit the result.
```

Git adds conflict markers in the file to show you both versions:

HTML

```
<<<<<< HEAD (main branch version)
<h1>Welcome to Naukri</h1>
=====
<h1>Welcome to Naukri Jobs Portal</h1>
>>>>>> feature-contact (your branch version)
```

Q21. How do you resolve a merge conflict in Git?

Answer: Resolving a conflict is a manual process — you read both versions and decide what the final code should look like.

Step-by-step:

Shell

```
# Step 1: See which files have conflicts
git status
# Both modified: index.html

# Step 2: Open the conflicted file and find the markers
# <<<<<< HEAD ... ===== ... >>>>>> branch-name

# Step 3: Edit the file - remove markers and keep what you want
# Example: you decide to keep the feature branch version
```

```
# <h1>Welcome to Naukri Jobs Portal</h1>

# Step 4: Stage the resolved file
git add index.html

# Step 5: Complete the merge with a commit
git commit -m "Merge feature-contact, resolve heading conflict"
```

Interview Tip: Interviewers ask "are you afraid of merge conflicts?" — the right answer is "no, they are a normal part of team development and the resolution process is straightforward." Knowing the exact steps shows experience.

Q22. What is `git diff`? How do you use it?

Answer: `git diff` shows exactly what lines changed in your files — line by line.

```
Shell
# See changes not yet staged (working directory vs staging)
git diff

# See changes that are staged (staging vs last commit)
git diff --staged

# Compare two branches
git diff main feature-login

# Compare two specific commits
git diff abc123 def456
```

Reading the output:

```
None
- <h1>Old Heading</h1>    ← lines removed (red, with minus)
+ <h1>New Heading</h1>    ← lines added (green, with plus)
```

Interview Tip: Always run `git diff --staged` before committing to double-check exactly what you are saving.

Q23. What is `git reset`? What does it do?

Answer: `git reset` is used to undo changes — either to unstage files or to undo commits.

Common use — unstage a file:

```
Shell
# You accidentally staged a file
git add wrong-file.html

# Remove it from staging (keep changes in working directory)
git reset wrong-file.html

# Modern equivalent
git restore --staged wrong-file.html
```

Three modes of `git reset` (on commits):

Mode	What it does
<code>--soft</code>	Moves commit back, keeps changes staged
<code>--mixed</code> (default)	Moves commit back, unstages changes
<code>--hard</code>	Moves commit back, deletes changes permanently ⚠

```
Shell
# Undo last commit but keep changes staged
git reset --soft HEAD~1

# Undo last commit, unstage changes (safe)
git reset HEAD~1

# ⚠ Danger - undo last commit AND delete all changes permanently
git reset --hard HEAD~1
```


Interview Tip: Never use `git reset --hard` on commits that have already been pushed to a shared repository — it rewrites history and causes problems for the whole team.

Q24. What is `git stash`? When would you use it?

Answer: `git stash` temporarily saves your uncommitted changes without making a commit, so you can switch to a different branch or task and come back to your work later.

Scenario: You are working on a new feature on the `feature-dashboard` branch. Your manager asks you to urgently fix a bug on `main`. Your feature is half done — you cannot commit it yet. You use `git stash`.

```
Shell
# Save work-in-progress without committing
git stash

# Check stash saved
git stash list
# stash@{0}: WIP on feature-dashboard: a3f9b21 Add card section

# Switch to main and fix the bug
git switch main
# ... fix the bug, commit it ...

# Switch back and restore your saved work
git switch feature-dashboard
git stash pop
```

`git stash pop` restores the saved changes and removes them from the stash.

Interview Tip: `git stash` is used constantly in real team development. Knowing this command shows practical experience.

Q25. What is the difference between `git stash` and `git commit`?

Answer:

	git stash	git commit
Purpose	Temporarily save incomplete work	Permanently save complete work
Appears in history?	✗ No	✓ Yes
Can be shared with team?	✗ No	✓ Yes (after push)
When to use	Quick pause, switching tasks	Feature or fix is complete

Shell

```
# Stash – temporary, not in log
git stash
git log  # stash does not appear here

# Commit – permanent, always in log
git commit -m "Add payment form"
git log  # commit appears here
```

Q26. What is the purpose of `git branch -d`? When would you use it?

Answer: `git branch -d` deletes a branch that is no longer needed — typically after it has been successfully merged into main.

Shell

```
# Safe delete – only deletes if branch is already merged
git branch -d feature-login

# Force delete – deletes even if not merged ⚠ (unmerged work is lost)
git branch -D feature-abandoned
```

Workflow in real companies:

Shell

```
# 1. Create feature branch
git switch -c feature-payment

# 2. Do work and commit
```

```
git add .  
git commit -m "Add payment gateway integration"
```

```
# 3. Merge into main  
git switch main  
git merge feature-payment
```

```
# 4. Delete the branch – work is done  
git branch -d feature-payment
```

Interview Tip: Keeping too many old branches is messy and confusing. Deleting merged branches is a good practice companies follow in code reviews.

Q27. A developer at Infosys committed some changes to the wrong branch. How do they move those commits to the correct branch?

Answer: Use `git cherry-pick` to copy a specific commit from one branch to another.

Shell

```
# Step 1: Note the commit ID from the wrong branch  
git log --oneline  
# a3f9b21 Add payment form ← this commit is on the wrong branch
```

```
# Step 2: Switch to the correct branch  
git switch correct-branch
```

```
# Step 3: Cherry-pick that commit onto the correct branch  
git cherry-pick a3f9b21
```

```
# Step 4: Go back and remove from wrong branch if needed  
git switch wrong-branch  
git reset --soft HEAD~1 # undo the commit, keep changes
```

Interview Tip: `git cherry-pick` is an intermediate-level Git concept — knowing it shows practical experience beyond just the basics.

Q28. What does `HEAD` mean in Git?

Answer: **HEAD** is a pointer that shows which commit you are currently on — it always points to the latest commit of your current branch.

Shell

```
# HEAD points to your current position
git log --oneline
# a3f9b21 (HEAD -> main) Add responsive navbar ← HEAD is here
# 7bc1d45 Initial project setup
```

HEAD~1 means "one commit before HEAD" — the previous commit. **HEAD~2** means two commits before the current one.

Shell

```
# Undo last commit
git reset HEAD~1

# See the previous commit's changes
git diff HEAD~1 HEAD
```

Interview Tip: Many Git commands use **HEAD** as a reference point. Understanding it makes **git reset**, **git diff**, and **git revert** much easier to understand.

Q29. What is the difference between **git merge and **git rebase**?**

Answer:

Feature	git merge	git rebase
History	Keeps full history, creates merge commit	Rewrites history, cleaner linear view
Safe for shared branches?	✅ Yes	❌ No — never rebase shared branches
Use case	Standard team workflow	Clean up local commits before pushing

Shell

```
# Merge – creates a merge commit, preserves full history
git switch main
git merge feature-ui
# History: A → B → C ← merge commit

# Rebase – replays feature commits on top of main
git switch feature-ui
git rebase main
# History: A → B → C (linear, no merge commit visible)
```

Interview Tip: Most companies use `git merge` for team workflows. `git rebase` is used to clean up personal branches before opening a pull request. Never rebase branches that others are using.

Q30. What is a pull request (PR)? How does it relate to Git branching?

Answer: A pull request is a request to merge your feature branch into the main branch. It is not a Git command — it is a feature of platforms like GitHub, GitLab, and Bitbucket.

Workflow:

Shell

```
# 1. Create a feature branch
git switch -c feature-search

# 2. Make changes and commit
git add .
git commit -m "Add search functionality to navbar"

# 3. Push branch to GitHub
git push origin feature-search

# 4. On GitHub – open a Pull Request
# → Team reviews your code
# → Reviewer approves or requests changes
# → Branch gets merged into main

# 5. Delete the branch locally after merge
git branch -d feature-search
```

Why PRs matter:

- Code is reviewed before it reaches production
- Team members learn from each other
- Every merge is documented with context

Interview Tip: All Indian IT companies — TCS, Infosys, Wipro, Zoho — and all startups use pull requests for code review. Interviewers will ask "have you worked with pull requests?" — always say yes and explain this flow.

Q31. What is the difference between `git fetch` and `git pull`?

Answer:

	<code>git fetch</code>	<code>git pull</code>
What it does	Downloads changes from remote — does NOT merge	Downloads AND merges into your current branch
Safe?	✅ Yes — nothing changes in your working files	⚠️ Can cause conflicts if your branch also has changes
Use when	You want to see what changed remotely first	You want to update your local branch immediately

Shell

```
# fetch - just download, don't merge yet
git fetch origin
git log origin/main --oneline # see what's on remote
git merge origin/main        # merge when you're ready

# pull - download and merge in one step
git pull origin main
```

Interview Tip: Senior developers prefer `git fetch` followed by `git merge` because they want to review changes before merging. `git pull` is fine for simple personal projects.

Q32. A developer accidentally deleted a file and committed that deletion. How do they get the file back?

Answer:

Shell

```
# Step 1: Find the commit where the file was deleted
git log --oneline -- deleted-file.html

# Step 2: Get the file back from the commit before it was deleted
git checkout <commit-id>~1 -- deleted-file.html

# Step 3: Stage and commit the restored file
git add deleted-file.html
git commit -m "Restore accidentally deleted file"
```

Alternative — if the deletion was the very last commit:

Shell

```
# Undo the last commit, keep changes staged
git reset --soft HEAD~1

# The deleted file will now show as "deleted but staged"
# Simply discard the deletion
git restore deleted-file.html

# Commit only the other changes if any
```

Q33. What is `git remote`? How do you connect a local repository to GitHub?

Answer: A remote is a version of your repository stored on another server (like GitHub). `git remote` manages these connections.

Shell

```
# Step 1: Create repo on GitHub (through the website)

# Step 2: Connect your local repo to GitHub
git remote add origin https://github.com/username/my-project.git
```

```
# Step 3: Check the remote was added
git remote -v
# Output:
# origin  https://github.com/username/my-project.git (fetch)
# origin  https://github.com/username/my-project.git (push)

# Step 4: Push your code to GitHub
git push -u origin main
```

`origin` is just a nickname for the remote URL — it is the standard default name used everywhere.

Q34. What is the standard Git workflow used in most companies?

Answer: Most companies follow a workflow called **Feature Branch Workflow**:

```
Shell

# 1. Start from an updated main branch
git switch main
git pull origin main

# 2. Create a feature branch
git switch -c feature-user-profile

# 3. Make changes and commit regularly
git add .
git commit -m "Add profile picture upload"

git add .
git commit -m "Add profile bio section"

# 4. Push branch to remote
git push origin feature-user-profile

# 5. Open a pull request on GitHub for code review

# 6. After approval, merge to main

# 7. Delete the feature branch
git branch -d feature-user-profile
```



```
git push origin --delete feature-user-profile
```

Interview Tip: Being able to explain this full workflow clearly in an interview — without needing to think too hard — shows you have used Git in real projects.

Q35. What are some common Git mistakes freshers make in their first job?

Answer:

1. Committing directly to main instead of using a branch:

```
Shell
# ❌ Bad practice
git switch main
# ... make changes ...
git commit -m "new feature"

# ✅ Good practice
git switch -c feature-new
# ... make changes ...
git commit -m "new feature"
# then open a pull request
```

2. Not committing often enough: Committing once a day with everything is too big. Commit after each small, logical change.

3. Vague commit messages:

```
Shell
# ❌ Bad
git commit -m "stuff"

# ✅ Good
git commit -m "Fix mobile menu not closing after link click"
```

4. Pushing `.env` files to GitHub: Always add `.env` to `.gitignore` before first commit.

5. Using `git reset --hard` without understanding it: This permanently deletes changes. Many freshers lose hours of work this way.

Q36. What is `git revert`? How is it different from `git reset`?

Answer:

	<code>git reset</code>	<code>git revert</code>
What it does	Moves history back — as if commits never happened	Creates a new commit that undoes a previous commit
Rewrites history?	✅ Yes	❌ No — history is preserved
Safe on shared branches?	❌ No	✅ Yes
Use when	Undoing local commits not yet pushed	Undoing commits already pushed/shared

Shell

```
# git reset - local only, rewrites history
git reset HEAD~1

# git revert - safe for shared branches, creates undo commit
git revert a3f9b21
# Creates a new commit: "Revert 'Add payment form'"
```

Interview Tip: In a team environment, always use `git revert` to undo pushed commits. Using `git reset` on shared history causes major problems for teammates.

Q37. What is `git log --oneline --graph`? When is it useful?

Answer: This command shows a visual representation of your branch and merge history in the terminal.

Shell

```
git log --oneline --graph
```

Output example:

None

```
* d7a3f21 (HEAD -> main) Merge feature-payment into main
| \
| * a1b2c3d (feature-payment) Add payment gateway
| * 9f8e7d6 Add payment form UI
* | c5d4e3f Update homepage header
|/
* 7bc1d45 Initial project setup
```

This visual helps you understand:

- Which branches were created
- Where merges happened
- The full timeline of changes

Used by developers to:

- Debug complex merge histories
- Understand what a colleague did
- Review the project timeline before a release

Q38. A team at a Pune startup has 3 developers all pushing to the same `main` branch. What problems will they face and how should they fix this?

Answer:

Problems with pushing directly to main:

- Constant merge conflicts as everyone edits the same files
- Broken code gets pushed directly — no review
- Hard to track who introduced a bug
- One person's push can overwrite another's work

Correct approach — Feature Branch Workflow:

Shell

```
# Developer 1 - works on their own branch
git switch -c feature-search
# ... work ... commit ... push ... PR → merge

# Developer 2 - works on their own branch
git switch -c feature-cart
# ... work ... commit ... push ... PR → merge

# Developer 3 - works on their own branch
git switch -c feature-auth
# ... work ... commit ... push ... PR → merge
```

Each developer works in isolation. Code only enters `main` after a pull request and review. No one blocks anyone else, and broken code never reaches production directly.

Q39. What is `git clone`? How is it different from `git init`?

Answer:

	<code>git init</code>	<code>git clone</code>
Use when	Starting a brand new project	Copying an existing repository
Downloads existing history?	✗ No	✓ Yes — full history included
Connects to remote automatically?	✗ No — you add remote manually	✓ Yes — remote is set up automatically

Shell

```
# git init - new project from scratch
mkdir my-project
cd my-project
git init

# git clone - copy existing repo from GitHub
git clone https://github.com/username/existing-project.git
cd existing-project
# you now have the full project with all history
```

Real use: On your first day at a new company, you `git clone` the company's repository to get the full project on your laptop.

Q40. How do you see which branches exist in your repository?

Answer:

```
Shell
# See local branches only
git branch
# Output:
# * main          ← asterisk = current branch
#   feature-login
#   feature-payment

# See remote branches only
git branch -r
# Output:
# origin/main
# origin/feature-login

# See all branches (local + remote)
git branch -a
```

Delete a branch after work is done:

```
Shell
# Local delete
git branch -d feature-login

# Remote delete
git push origin --delete feature-login
```

Q41. What happens when you run `git merge` and Git shows "Already up to date"?

Answer: This message means the branch you are trying to merge is not ahead of the current branch — all its commits are already included. There is nothing new to merge.

Shell

```
git switch main
git merge feature-login
# Output: Already up to date.
```

This happens when:

- The feature branch has not had any new commits since it was created or last merged
- You accidentally merged the branch already and are trying to do it again
- You are on the feature branch and trying to merge itself

This is not an error — it just means the branches are already in sync. No action is needed.

Q42. What is the difference between `git switch` and `git checkout`?

Answer: Both switch branches, but `git switch` is the modern, cleaner command introduced in Git 2.23 (2019). `git checkout` was the original command but it did too many things — switching branches, restoring files, and creating branches all with the same command.

Shell

```
# Switching branches
git checkout main          # old way
git switch main            # new, preferred way

# Creating and switching
git checkout -b feature-1  # old way
git switch -c feature-1    # new way

# Restoring a file (nothing to do with branches)
git checkout -- file.html  # old way (confusing)
git restore file.html      # new way (cleaner)
```

Interview Tip: Both commands work. Most companies accept either. But knowing `git switch` and `git restore` shows you are up to date with modern Git practices.

Q43. A developer pushed a commit with a password accidentally included in a config file. What should they do?

Answer: This is a serious security issue. The steps are:

Immediate actions:

1. **Revoke the password/API key immediately** — go to whichever service issued it (AWS, Google, Razorpay, etc.) and disable or regenerate the key. This is the most important step — even if you delete the commit, the key may already be compromised.
2. **Remove the secret from the file and commit:**

Shell

```
# Remove the secret from the file
# Add the file to .gitignore
echo "config.js" >> .gitignore

git add config.js .gitignore
git commit -m "Remove accidentally committed credentials"
git push
```

3. **To remove from Git history** (advanced — requires `git filter-branch` or BFG Repo Cleaner):

Shell

```
# This rewrites full history - coordinate with team
git filter-branch --force --index-filter \
  "git rm --cached --ignore-unmatch config.js" \
  --prune-empty --tag-name-filter cat -- --all
```

Interview Tip: The right first answer is always: "Revoke the credential immediately." Removing the commit history alone is not enough because the key was already exposed.

Q44. What is a Git tag? How is it used in real projects?

Answer: A Git tag marks a specific commit as important — usually to mark a release version.

Shell

```
# Create a tag for version 1.0 release
```

```
git tag v1.0

# Create an annotated tag with a message (preferred)
git tag -a v1.0 -m "First production release"

# View all tags
git tag

# Push tags to GitHub
git push origin --tags
```

Real use: After Flipkart deploys a new version of their mobile website, they tag that commit as `v2.4.1`. If a bug appears, they can instantly see what code was deployed using the tag — without searching through hundreds of commits.

Interview Tip: Tags are commonly used in companies that do scheduled releases. Understanding tags shows you understand the deployment cycle.

Q45. Walk through a complete, real-world Git workflow — from starting a feature to getting it into the main branch.

Answer: This is a common interview question where they want you to walk through everything end to end.

```
Shell

# Step 1: Update your local main branch
git switch main
git pull origin main

# Step 2: Create a feature branch
git switch -c feature-job-search

# Step 3: Write your code, then check what changed
git status
git diff

# Step 4: Stage your changes
git add search.html search.css

# Step 5: Commit with a meaningful message
```



```
git commit -m "Add job search bar with location filter"

# Step 6: Continue working and commit regularly
git add search.js
git commit -m "Add search autocomplete functionality"

# Step 7: Before pushing, update from main to catch any new changes
git fetch origin
git merge origin/main
# Resolve any conflicts if they appear

# Step 8: Push your branch to GitHub
git push origin feature-job-search

# Step 9: Open a Pull Request on GitHub
# → Add description of what you built
# → Request a code review from a teammate

# Step 10: Reviewer approves → merge into main on GitHub

# Step 11: Clean up locally
git switch main
git pull origin main          # get the merged code
git branch -d feature-job-search # delete local branch
```

This full workflow is used daily at companies like Naukri, Zoho, Razorpay, and every startup in India with a development team.